Cahier des Charges Fonctionnel & Technique : StoreMaster React

---

Projet d'apprentissage approfondi de React (Vite, Tailwind CSS, Hooks, Router, Context & API)

> **Objectif global :** Concevoir une Single Page Application (SPA) e-commerce complète permettant d'explorer un catalogue de produits via une API externe, de filtrer/rechercher des articles, de consulter une fiche détaillée et de gérer un panier d'achat réactif avec persistance locale.

---

1. Stack Technique & Écosystème

Outil / Bibliothèque	Rôle	Motivation pédagogique
React 18 / 19	Bibliothèque UI	Composants fonctionnels, JSX, Hooks (useState, useEffect, useContext).
Vite	Bundler & Serveur Dev	Démarrage instantané, Hot Module Replacement (HMR) et compilation rapide.
Tailwind CSS	Framework CSS utilitaire	Stylisation moderne,

Outil / Bibliothèque	Rôle	Motivation pédagogique
		responsive et déclarative sans quitter le JSX.
React Router DOM (v6+)	Routage SPA	Navigation sans rechargement, paramètres dynamiques (/product/:id).
FakeStore API	Backend REST public	Appels asynchrones, gestion des loaders, erreurs réseau et données réalistes.

---

2. Arborescence Exhaustive du Projet

```
storemaster-react/
├── node_modules/              # Dépendances téléchargées (géré par npm)
├── public/                    # Fichiers statiques servis sans transformation
│   └── favicon.svg
├── src/                       # Code source de l'application
│   ├── assets/                # Médias importés (logos, icônes)
│   ├── components/            # Composants réutilisables et modulaires
│   │   ├── Navbar.jsx         # Barre de navigation avec badge panier dynamique
│   │   ├── ProductCard.jsx    # Carte unitaire de produit (image, prix, note, bouton)
│   │   ├── ProductList.jsx    # Grille responsive affichant les cartes produits
│   │   ├── SearchBar.jsx      # Champ de recherche et filtres par catégories
│   │   ├── CartItem.jsx       # Ligne de produit dans le panier (quantité, suppression)
│   │   └── Loader.jsx         # Spinner / Skeleton pendant le chargement des données
│   ├── pages/                 # Vues associées aux différentes routes
```

```
│   │   ├── HomePage.jsx      # Page catalogue principale
│   │   ├── ProductDetail.jsx # Page détail d'un produit (route /product/:id)
│   │   ├── CartPage.jsx      # Page récapitulative du panier et passage de commande
│   │   └── NotFound.jsx      # Page d'erreur 404
│   ├── context/              # Gestion d'état global
│   │   └── CartContext.jsx   # Provider & Custom Hook (useCart) pour le panier
│   ├── services/             # Fonctions d'interaction API (Fetch)
│   │   └── api.js            # Requêtes API centralisées
│   ├── App.jsx               # Définition des routes et structure de base
│   ├── index.css             # Directives Tailwind (@tailwind base, components,
utilities)
│   └── main.jsx              # Point d'entrée React (ReactDOM.createRoot sur #root)
├── .env                      # Variables d'environnement (ex: VITE_API_URL)
├── .gitignore                # Fichiers exclus de Git (node_modules, dist, .env)
├── index.html                # Seule page HTML avec <div id="root"></div>
├── package.json              # Dépendances et scripts npm (dev, build, preview)
├── package-lock.json         # Arbre des versions verrouillées
├── postcss.config.js         # Configuration du pipeline CSS pour Tailwind
├── tailwind.config.js        # Configuration des thèmes et chemins de fichiers JSX
└── vite.config.js            # Configuration de Vite avec plugin @vitejs/plugin-react
```

---

3. Modèle de Données (Types & Structures)

Structure d'un Produit (issu de FakeStore API)

```
{
  "id": 1,
  "title": "Fjallraven - Foldsack No. 1 Backpack",
  "price": 109.95,
  "description": "Your perfect pack for everyday use...",
  "category": "men's clothing",
  "image": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
  "rating": {
```

```
    "rate": 3.9,
    "count": 120
  }
}
```

Structure d'un Article dans le Panier (`CartItem`)

```
{
  "id": 1,
  "title": "Fjallraven - Foldsack No. 1 Backpack",
  "price": 109.95,
  "image": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
  "quantity": 2
}
```

---

4. Spécifications Fonctionnelles Détaillées

F1 : Récupération & Affichage du Catalogue

- Au montage du composant HomePage, déclencher un effet de bord via useEffect pour récupérer les produits depuis l'API (GET https://fakestoreapi.com/products).
- Afficher un composant Loader pendant la durée de la requête réseau.
- Intercepter et afficher un message clair en cas d'erreur de chargement (bloc try / catch).
- Afficher les articles dans une grille CSS responsive via ProductList et ProductCard en utilisant la méthode .map() avec une clé unique key={product.id}.

F2 : Recherche & Filtrage Multi-critères

- Barre de recherche textuelle avec contrôle d'état React (value={searchTerm}, onChange).
- Boutons ou menu déroulant pour filtrer par catégorie (vêtements hommes, femmes, bijoux, électronique).

- Filtrage combiné réactif calculé dynamiquement (sans recharger l'API).

F3 : Page Fiche Produit (Routage Dynamique)

- Au clic sur une carte, redirection vers /product/:id sans recharger la page (via Link de React Router).
- Extraction du paramètre d'URL grâce au hook useParams().
- Récupération des détails du produit spécifique (GET /products/:id).
- Affichage de l'image agrandie, note moyenne, avis, description complète et bouton d'ajout au panier.

F4 : Gestion du Panier (CartContext)

- L'état du panier doit être global et accessible depuis n'importe quel composant sans "prop drilling".
- **Ajout au panier (addToCart) :** Si l'article existe déjà, incrémenter sa quantity de façon immuable ; sinon, l'ajouter avec quantity: 1.
- **Modification de la quantité :** Boutons + et - pour ajuster le nombre d'exemplaires. Si la quantité tombe à 0, supprimer automatiquement l'article.
- **Suppression unitaire (removeFromCart) :** Supprimer l'article par son identifiant avec .filter().
- **Vider le panier (clearCart) :** Réinitialiser le panier à [].
- **Badge dynamique dans la Navbar :** Affiche en temps réel le total d'articles cumulés (calculé via .reduce()).

F5 : Persistance des Données (LocalStorage)

- À chaque modification de l'état du panier, synchroniser les données dans le localStorage via un hook useEffect dédié.
- Au premier démarrage, initialiser l'état du panier avec les données stockées s'il en existe.

---

5. Configuration des Fichiers Clés de Démarrage

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    open: true
  }
});
```

`tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

`src/index.css`

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

```
body {
  @apply bg-gray-50 text-gray-800 antialiased;
}
```

---

6. Critères d'Acceptation & Validation

- ---

  [ ] Navigation fluide entre /, /product/:id et /cart sans aucun rechargement complet de l'onglet.
- [ ] Actualiser la page panier (F5) ne perd aucun article grâce au localStorage.
- [ ] Les données du catalogue proviennent bien d'un appel asynchrone réel avec indicateur visuel de chargement.
- [ ] Aucune mutation directe de state n'est réalisée (tous les setters utilisent des copies immuables).
- [ ] Le design est responsive sur mobile, tablette et desktop grâce aux classes Tailwind CSS.